Library Management System – Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This document provides a detailed description of the requirements, scope, architecture, and functionality of the Library Management System. The system is intended to digitize and streamline the management of library resources, including books, users, and transactions, while ensuring scalability, security, and maintainability.

1.2 Scope

The Library Management System is a full-stack web application designed to manage both digital and physical book inventories. The system enables users to browse, search, and request books, while administrators manage inventory, issue/return operations, and user activities.

The system will include authentication, role-based access control, book management, transaction tracking, notification mechanisms, and reporting capabilities.

1.3 Objectives

- Reduce manual effort in managing library operations
- Provide real-time tracking of book availability
- Enable secure and role-based access to system features
- Improve user experience through efficient search and access
- Maintain accurate records of book transactions and users

---

2. Problem Statement

Traditional library systems rely heavily on manual processes or outdated digital systems that lack real-time tracking, scalability, and user-friendly interfaces. This leads to inefficiencies such as:

- Difficulty in tracking book availability
- Lack of transparency in issue and return processes
- Inefficient user management
- Absence of automated reminders and fine calculations
- Limited accessibility for users to explore available resources

The proposed system aims to address these challenges by providing a centralized, automated, and scalable platform.

---

3. System Overview

3.1 Users

The system will have two primary user roles:

1. Admin (Librarian)
    - Manages books, users, and transactions
    - Updates inventory and availability
    - Monitors issued books and overdue returns
2. User (Member)
    - Registers and logs into the system
    - Searches and browses books
    - Views availability and requests books

---

4. Technology Stack

Backend

- Java
- Spring Boot
- Spring Data JPA
- Spring Security with JWT

Frontend

- Angular

Database

- MySQL

Caching

- Redis

Storage

- AWS S3 for storing digital book files

Tools

- Postman for API testing
- Git for version control

---

5. System Architecture

5.1 Initial Architecture

The system will initially follow a modular monolithic architecture:

- Controller Layer
- Service Layer
- Repository Layer
- DTO Layer

This approach ensures simplicity during development while maintaining clean separation of concerns.

5.2 Future Architecture

The system can be evolved into a microservices architecture with separate services such as:

- Authentication Service
- Book Service
- Transaction Service
- Notification Service

---

6. Functional Requirements

6.1 Authentication Module

- User registration with email and password
- OTP verification via email (SMTP)
- Secure login using JWT authentication
- Role-based access control (Admin/User)

6.2 Book Management Module

- Add, update, and delete books (Admin only)
- Maintain book details including:
    - ISBN
    - Title
    - Author
    - Genre
    - Description
    - Offline count
    - Location (floor, rack)
    - Digital file URL

6.3 Search and Browse Module

- Search books by title, author, or genre
- Filter and pagination support
- Genre-based categorization

6.4 Book Issue and Return Module

- Issue books to users (Admin action)
- Track issue date, due date, and return date
- Automatically update availability
- Reduce offline count on issue and increase on return

6.5 Notification Module

- Send email on book issue
- Send reminder email before due date
- Notify users of overdue books

6.6 Fine Management

- Calculate fines for overdue returns
- Configurable fine per day

6.7 Member Management

- View all registered users (Admin)

- Track borrowing history
- View active and past transactions

6.8 Book Request Module

- Users can request unavailable books
- Admin can view and manage requests

6.9 Dashboard Module

- Display summary metrics:
    - Total books
    - Issued books
    - Available books
    - Active users

6.10 Static Pages

- About page describing the application
- Developer information page with contact details

---

7. Non-Functional Requirements

7.1 Performance

- System should handle concurrent users efficiently
- Use caching (Redis) for frequently accessed data

7.2 Security

- JWT-based authentication
- Password encryption
- Role-based authorization

7.3 Scalability

- Designed to support migration to microservices
- Modular structure for easy extension

7.4 Reliability

- Ensure data consistency in transactions

- Proper error handling and logging

---

8. Database Design (High-Level)

The system will include the following tables:

- Users
- Roles
- Books
- Genres
- Transactions
- Fines
- Requests

Each table will be designed with proper relationships to ensure normalization and efficient querying.

---

9. Development Approach

The development will follow an incremental and modular approach:

1. Requirement finalization
2. Backend setup and core APIs
3. Authentication and security implementation
4. Book management and search functionality
5. Transaction and notification modules
6. Frontend integration
7. Testing and deployment

Each module will be developed, tested, and validated before moving to the next.

---

10. Future Enhancements

- Migration to microservices architecture
- Integration with external APIs for book metadata

- Advanced analytics and reporting
- Mobile application support

---

11. Conclusion

This project aims to build a robust, scalable, and user-friendly Library Management System using modern technologies. The system is designed not only to fulfill current requirements but also to accommodate future enhancements and scaling needs.

The structured approach ensures clarity in development, maintainability of code, and readiness for real-world deployment.